ASSIGNMENT 2 – TASK 3

THREADS: POSIX MULTITHREADING FOR A 64-CAMERA NUMBER-PLATE READER

1. Introduction and Problem Context

The assignment describes a city traffic system that receives number-plate frames from 64 cameras. The first version processes cameras sequentially and falls behind during peak hours. Task 3 requires POSIX-thread processing using one thread per camera and thread pools of 4, 8 and 16 threads. It also asks for an explanation of thread resources, thread models, measured speed-up, unprotected shared-counter updates and a process-based alternative.

The question does not provide camera images, a recognition model or a fixed frame workload. Therefore, the supplied program uses a deterministic synthetic CPU workload as a stand-in for frame analysis. It tests concurrent work distribution, not actual image decoding or number-plate recognition. A real deployment should replace this stand-in with the actual frame capture, preprocessing and recognition pipeline.

2. Aim and Objectives

Aim: To implement and evaluate concurrent camera-frame processing using POSIX threads and compare thread-per-camera execution with bounded worker pools.

Objective	What is demonstrated
Threading strategies	Sequential baseline, 64 threads, and pools of 4, 8 and 16 workers.
Thread resources	Shared process resources and each thread's private execution context.
Thread models	User-level and kernel-level threads; many-to-one, one-to-one and many-to-many.
Performance	Execution-time measurement and speed-up on the executing machine.
Synchronisation	Race conditions caused by unprotected shared-counter updates and mutex protection.
Alternative concept	Comparison of thread-based and process-based frame processing.

3. Workload and Design Assumptions

The benchmark processes the same synthetic workload in every configuration:

Parameter	Value
Number of cameras	64
Frames per camera	4
Synthetic work per frame	200,000 deterministic CPU iterations
Trials per configuration	5
Reported time	Median elapsed time across five trials

Each camera produces a deterministic checksum. The program verifies that the sequential baseline, one-thread-per-camera strategy and all three worker pools produce identical per-camera results. Thread creation and joining are included in threaded timings; this makes their overhead visible. The sequential baseline does not incur thread creation/join overhead.

4. Proposed Threading Architecture

4.1 Sequential baseline

The main thread processes camera IDs 0 through 63 in order. Its median execution time is the baseline for speed-up calculations.

4.2 One thread per camera

The program creates 64 POSIX threads. Each thread receives one camera ID, processes that camera's four synthetic frames and writes its checksum into the corresponding result-array element. The main thread joins all 64 workers before inspecting results.

4.3 Fixed worker pools

The program repeats the same workload with 4, 8 and 16 worker threads. Workers obtain the next camera index from a shared C11 atomic counter. Each worker processes the claimed camera and writes to a separate result slot. The atomic counter prevents two workers from claiming the same camera.

4.4 High-level flow

```
Start benchmark
 |-- Sequential: cameras 0 through 63
 |-- One thread per camera: create 64, then join 64
 |-- Pool of 4: workers claim cameras atomically
 |-- Pool of 8: workers claim cameras atomically
 |-- Pool of 16: workers claim cameras atomically
 |-- Verify checksums and calculate median times/speed-up
 |-- Run the mutex-protected counter test
Finish
```

5. Shared and Private Thread Resources

Shared within one process	Private to each thread
Program code / text segment	Thread stack
Global and static variables	CPU register state
Heap memory and dynamically allocated objects	Program counter / instruction pointer
Open file descriptors and process-level resources	Thread-local storage
Process address space and shared data structures	Thread scheduling / execution state

Threads in one process share the same address space, allowing communication through shared variables and data structures without an IPC mechanism. This is efficient, but unsynchronised updates can conflict. Each thread has its own stack, register context, program counter and thread-local storage.

6. User-Level and Kernel-Level Threads

Aspect	User-level threads (ULT)	Kernel-level threads (KLT)
Management	Managed by a user-space library/runtime.	Created and managed by the operating-system kernel.
Scheduling	A user runtime selects which user thread runs.	The kernel scheduler schedules kernel-visible threads.
Switching	May be inexpensive because management can occur in user space.	Requires kernel involvement and generally more bookkeeping.
Blocking call	In a pure many-to-one design, one blocking system call can block all user threads.	A blocked kernel thread does not necessarily block other threads in the process.
Parallel execution	Pure many-to-one cannot run multiple user threads simultaneously on separate cores.	Can use multiple CPU cores when multiple kernel threads are runnable.

These are general model distinctions; exact behaviour depends on the operating system and thread library. POSIX pthreads on common modern Linux systems are generally implemented as kernel-scheduled threads.

7. Thread Mapping Models

Model	Mapping	Strengths	Limitations
Many-to-One	Many user threads map to one kernel thread.	Low kernel-thread overhead; user library can schedule quickly.	No true multi-core parallelism; a blocking call can block all user threads.
One-to-One	Each user thread maps to one kernel thread.	True parallelism; blocking one thread need not block others.	More kernel objects and management overhead.
Many-to-Many	Many user threads are multiplexed over a set of kernel threads.	Combines user-level scheduling with kernel-level parallelism.	More complex runtime and scheduling coordination.

Many-to-One: U1, U2, U3, U4 → K1 → CPU

One-to-One: U1 → K1; U2 → K2; U3 → K3; U4 → K4 → CPU cores

Many-to-Many: U1, U2, U3, U4, U5 → K1, K2, K3 → CPU cores

For 64 cameras, one-to-one kernel-visible threading is straightforward with POSIX pthreads on Linux. However, creating 64 threads can impose more stack reservation, scheduling and creation/join overhead than a bounded pool. A pool of 4, 8 or 16 caps active concurrency; its size should be selected using measurements and available CPU cores.

8. Performance Measurement and Speed-Up

The program measures elapsed wall-clock time using clock_gettime(CLOCK_MONOTONIC). Each configuration runs five trials and reports the median, reducing the influence of an unusually fast or slow trial.

Speed-up = Sequential median execution time / Threaded median execution time

A speed-up greater than 1 indicates that the threaded version completed faster than the sequential baseline. A value below 1 indicates that thread-management overhead or contention outweighed parallel-processing benefits for this workload and machine.

8.1 Results table - populate from your local benchmark

Configuration	Median time (ms)	Speed-up
Sequential	From benchmark_results.txt	1.000x
One thread per camera (64)	From benchmark_results.txt	Calculated by program
Worker pool (4)	From benchmark_results.txt	Calculated by program
Worker pool (8)	From benchmark_results.txt	Calculated by program
Worker pool (16)	From benchmark_results.txt	Calculated by program

Measurement integrity: No hardware timing values are invented in this report. Run the program on the target Ubuntu/WSL machine and use its output. The program prints median time and speed-up for all five configurations.

8.2 Why speed-up stops increasing

Factor	Effect on scaling
CPU core limit	Once runnable threads exceed available cores, extra threads compete rather than add parallel capacity.
Thread creation and joining	Creating/joining many threads consumes time; this overhead is included in threaded timings.
Context switching	Switching between runnable threads consumes CPU time that does not advance frame processing.
Synchronisation	Locks, atomics and shared queues can serialize parts of the workload.
Memory bandwidth and cache	Workers can compete for cache capacity and memory bandwidth.
Work imbalance	Some camera tasks may finish later than others, leaving workers unevenly occupied.

The best pool size is hardware- and workload-dependent. Record CPU information using nproc and lscpu when reporting results.

9. Shared Counter and Race Condition

A statement such as shared_counter++ is a read-modify-write operation and is not guaranteed to be atomic. If two threads read the same old value before either writes its incremented value, one update can be lost. The final counter may be lower than the total increments. In standard C, unsynchronised concurrent writes to a non-atomic shared object constitute a data race and produce undefined behaviour; the exact incorrect result is not guaranteed.

Unsafe illustration (do not use in production):

```
shared_counter++;  /* no lock: data race */
```

Correct mutex-protected update:

```
pthread_mutex_lock(&counter_mutex);
++shared_counter;
pthread_mutex_unlock(&counter_mutex);
```

The supplied program includes a mutex-protected counter check using four threads, each performing 100,000 increments. The expected final count is 400,000. The program reports PASS when actual and expected match. The camera benchmark uses an atomic work index and writes each camera's result to a distinct array element.

10. Alternative Concept: Multiple Processes Instead of Threads

A process-based design could launch a fixed number of worker processes. Each worker would receive camera IDs or frame batches, process them in its own address space and return checksums or recognition results through a pipe, message queue, socket or shared-memory region. The parent would collect and combine the results.

Aspect	Threads	Processes
Address space	Threads share one process address space.	Each process has a separate virtual address space.
Creation	Thread creation is generally lighter.	Process creation usually has higher setup and memory-management overhead.
Switching	Switching within one process may avoid address-space changes; register/scheduler costs remain.	May require address-space and memory-management context changes; depends on OS/hardware.
Sharing results	Shared data structures; mutexes/atomics may be required.	IPC such as pipes, sockets, queues or explicit shared memory is needed.
Isolation	A serious memory error in one thread can corrupt the whole process.	Address-space isolation generally contains ordinary memory faults to one process.
Coordination	Shared variables are efficient but need synchronisation.	IPC adds communication and possibly copying/synchronisation overhead.

For CPU-bound work on one machine, a process pool can use multiple cores just as threads can, but it normally needs more memory and explicit IPC for results. Threads are natural when workers share read-only frame data and return small per-camera results. Processes may be preferred where stronger fault isolation is required or separate services are already used.

11. Time and Space Complexity

Measure	Thread implementation	Process implementation
Total work	$O(C \times F \times W)$, C=64 cameras, F=frames/camera, W=work/frame.	Same computational work: $O(C \times F \times W)$.
Ideal parallel time	$O(C \times F \times W / p)$ for p workers, ignoring overhead and assuming balanced work.	Same ideal compute bound with p worker processes.
Scheduling/coordination	Atomic work-index operation is $O(1)$ per camera claim; total $O(C)$.	IPC dispatch/result operations add communication and coordination costs.
Additional storage	$O(C)$ result array plus $O(p)$ worker contexts/stacks.	$O(C)$ results plus $O(p)$ process state/private address-space overhead; IPC buffers add storage.

The asymptotic computational workload does not become smaller merely because threads or processes are used. Parallelism can reduce elapsed wall-clock time, but creation, switching, synchronisation, scheduling, IPC and workload imbalance contribute overhead. Big-O describes growth, not machine-specific elapsed time.

12. Evaluation and Design Recommendation

For this traffic-camera scenario, a bounded worker pool is a strong initial design because it avoids creating an unbounded number of threads and provides controllable concurrency. The recommended pool size should be the configuration that gives the best measured throughput and acceptable responsiveness on the target server. For CPU-bound work, test around the available CPU-core count and compare 4, 8 and 16 as required. For I/O-bound camera capture, useful concurrency may be higher. If a recognition library uses internal threads, account for oversubscription.

The final choice must be defended with measured medians, calculated speed-ups, system CPU information and fault-isolation requirements. No specific pool size is claimed to be universally optimal.

13. Conclusion

POSIX threads can process the 64-camera workload concurrently. One thread per camera exposes per-camera concurrency but incurs the cost of creating and joining 64 threads. Fixed pools of 4, 8 and 16 workers limit active threads and distribute work dynamically. Threads share the process address space but retain private stacks and register state; shared mutable data must be protected with mutexes or atomic operations. Thread models differ in blocking, parallelism and overhead. A process-based alternative improves isolation but requires separate address spaces and IPC for combining results. Select the

final worker-pool size using measurements on the actual target machine.

14. Execution Procedure and Evidence

1. Save camera_threads.c and run_task3.sh in the same Task03 folder.
2. Open Ubuntu/WSL in that folder.
3. Execute: chmod +x run_task3.sh
4. Execute: ./run_task3.sh
5. The script compiles with GCC and -pthread, runs all configurations and saves benchmark_results.txt.
6. Record gcc --version, lsb_release -a, uname -r, nproc and lscpu.
7. Capture a screenshot of the benchmark table, checksum verification and mutex-counter PASS result.

Evidence checklist: completed C source; successful compile/run; measured times for sequential, 64-thread, 4-thread, 8-thread and 16-thread configurations; speed-up values; checksum verification; mutex counter result; system/compiler details; and a reasoned thread-model evaluation.

15. Interpreting the Local Benchmark

Read the printed median times rather than relying on a single trial. A lower median indicates higher measured throughput for this fixed synthetic workload. A speed-up below 1.0x means the threaded configuration was slower than the sequential baseline. Identical-checksum verification confirms that each configuration processed the same deterministic tasks. The mutex-counter PASS result confirms that the protected counter reached its expected total; it does not imply that unsynchronised increments are safe.

The benchmark includes thread creation and joining in threaded elapsed time. It does not measure actual camera capture latency, image decoding, plate-recognition accuracy, GPU acceleration or network I/O. Those should be measured separately when a real camera-recognition pipeline is available.

Source alignment: This report addresses Q3 – Threads, 6 Marks, K5 – Evaluate, on page 14 of the supplied Assignment 2 question PDF.